

MandiMitra

Smart Slot Booking & Queue Management for Mandi Procurement

Smart India Hackathon 2026 — Detailed Project Report

Problem Statement ID: **26032**

Ministry of Consumer Affairs, Food & Public Distribution

Department of Consumer Affairs (DoCA)

Category: Software | Theme: Smart Automation

Live application: mandi-mitra-eosin.vercel.app

Team Name: *[add here]*

Institute: *[add here]*

September 2026

Table of Contents

1. Problem Statement
2. Proposed Solution
3. System Architecture
4. Database Design
5. Technology Stack
6. Module-wise Description
7. API Reference
8. How MandiMitra Meets Each SIH Requirement
9. Feasibility and Viability
10. Testing and Validation
11. Impact and Benefits
12. Current Implementation Status
13. Future Scope
14. References
15. Team Details

1. Problem Statement

Field	Detail
Problem Statement ID	26032
Title	Farmers often face long waiting times, lack of information regarding procurement schedules, and uncertainty about procurement status.
Organization	Ministry of Consumer Affairs, Food & Public Distribution
Department	Department of Consumer Affairs (DoCA)
Category	Software
Theme	Smart Automation

SIH's expected solution is a platform that:

- 1 Enables farmer registration and slot booking
- 2 Provides real-time queue management
- 3 Sends SMS/app notifications
- 4 Tracks procurement and payment status
- 5 Reduces congestion and waiting time at procurement centres

2. Proposed Solution

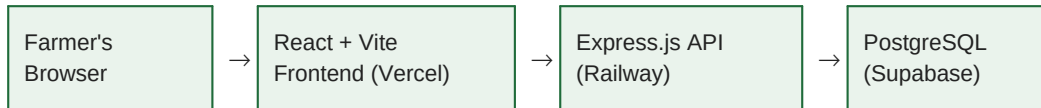
MandiMitra is a bilingual (12 Indian languages) web platform that gives every farmer a real, logged-in account to book a mandi procurement slot in advance, track their live position in the queue, see live crop prices and weather, and get help from an AI assistant — replacing the current system of showing up and waiting with no visibility into timing or status.

A farmer registers once with phone number, state, and district. From their dashboard they can:

- Book a slot for a specific crop, mandi, quantity, and date
- Track the live status of that booking as it moves through the real procurement pipeline (confirmed → arrived → graded → payment initiated → paid)
- See their live position in the queue for a given day at a given mandi
- Check live government mandi prices (data.gov.in) and MSP for their crop before deciding where/when to sell
- Check live weather at their chosen mandi
- Talk to an AI chatbot (in their own language) that can book a slot, check a booking's status, check queue position, check MSP/price, check weather, or file/check a grievance through natural conversation
- File and track grievances tied to a specific booking
- Edit their profile, log out, or permanently delete their account

3. System Architecture

The frontend is a single-page React app that talks to the backend only over authenticated REST calls — it holds no business logic or direct database access. The backend is a single Express service that owns all business rules (booking creation, queue position, status transitions, auth) and is the only component that talks to the database or any external API, so credentials for Supabase, Gemini, OpenWeatherMap, and data.gov.in live only on the server. Authentication uses a signed JWT (30-day expiry) issued on login/register and sent as a Bearer token on every request that needs a logged-in farmer.



The Express API also calls out, server-side, to data.gov.in (live crop prices), OpenWeatherMap (live weather and geocoding), and the Google Gemini API (the AI chatbot's function calling).

Request lifecycle example — booking a slot

- 1 Farmer fills the booking form on the Find Mandi page and submits
- 2 Frontend sends POST /bookings with crop, mandi, quantity, and date
- 3 Backend generates a unique token number (format MM-YYYY-NNNN), creates the row via Prisma, and returns the booking with crop and mandi details joined in
- 4 Frontend shows the confirmed booking and token to the farmer immediately

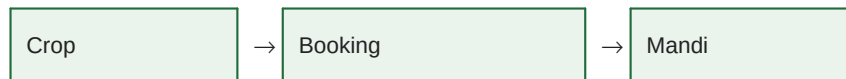
Deployment topology

Layer	Service	Notes
Frontend	Vercel	Builds the Vite app on every push to main; serves static assets globally
Backend	Railway	Runs the Express API as a long-lived Node process
Database	Supabase (PostgreSQL)	Accessed through Supabase's Transaction Pooler for IPv4 compatibility
Source control	GitHub	Both Vercel and Railway auto-deploy from the same repository's main branch

4. Database Design

MandiMitra uses a single PostgreSQL database with five tables, managed through the Prisma ORM. Bookings and Grievances store the farmer's name and phone as plain fields rather than a foreign key to Farmer — mirroring how a real mandi procurement record works (a paper token isn't erased if the farmer's account later changes), and meaning a farmer can delete their account without losing the procurement history tied to their bookings.

Entity relationships



Booking ← Grievance (optional, one booking can have several grievances)

Farmer is a standalone table (no foreign key from Booking or Grievance).

Farmer

Field	Type	Notes
id	Int	Primary key, auto-increment
name	String	Full name
phone	String	Unique; used as login identifier
passwordHash	String	bcrypt hash (10 salt rounds); raw password never stored
state	String	
district	String	
preferredLanguage	String	Default "hi"; drives the UI language on login
createdAt	DateTime	Default now()

Crop

Field	Type	Notes
id	Int	Primary key
nameEn / nameHi	String	English and Hindi display names
slug	String	Unique identifier
mSPPerQuintal	Float?	Government Minimum Support Price, optional
createdAt	DateTime	Default now()

Mandi

Field	Type	Notes
id	Int	Primary key
nameEn / nameHi	String	English and Hindi display names
slug	String	Unique identifier
district / state	String	
latitude / longitude	Float?	Filled by live geocoding on bulk import; used for live weather lookup

Field	Type	Notes
createdAt	DateTime	Default now()

Booking

Field	Type	Notes
id	Int	Primary key
tokenNumber	String	Unique, generated as MM-YYYY-NNNN
farmerName / farmerPhone	String	Snapshot of the farmer at booking time
cropId → Crop	Int (FK)	
mandiId → Mandi	Int (FK)	
quantityQuintal	Float	
slotDate	DateTime	Requested procurement date
status	String	CONFIRMED → ARRIVED → GRADED → PAYMENT_INITIATED → PAID, or CANCELLED
createdAt	DateTime	Default now(); also used to order the live queue

Grievance

Field	Type	Notes
id	Int	Primary key
ticketNumber	String	Unique, generated as GR-YYYY-NNNN
farmerName / farmerPhone	String	
subject / description	String	
status	String	OPEN → IN_PROGRESS → RESOLVED, or REJECTED
bookingId → Booking	Int? (FK)	Optional — a grievance need not be tied to a booking
createdAt / updatedAt	DateTime	

5. Technology Stack

Layer	Technology
Frontend framework	React (Vite build tooling)
Frontend routing	React Router
Frontend styling	Tailwind CSS
Frontend animation	Framer Motion
Icons	Lucide React
Internationalization	i18next (12 Indian languages)
Backend framework	Node.js + Express.js
ORM	Prisma
Database	PostgreSQL (hosted on Supabase, via Transaction Pooler)
Authentication	JWT (jsonwebtoken, 30-day expiry) + bcrypt password hashing
AI chatbot	Google Gemini API with function calling (7 real functions — see Module-wise Description)
Live mandi prices	data.gov.in open government data API
Live weather & geocoding	OpenWeatherMap API
Frontend hosting	Vercel
Backend hosting	Railway
Version control / CI	GitHub (push-to-deploy on both Vercel and Railway)

6. Module-wise Description

6.1 Authentication & Account Module

Handles registration, login, session verification, profile update, and permanent account deletion. Passwords are hashed with bcrypt before storage; a successful login/register issues a JWT containing the farmer's id, valid for 30 days. Every protected route checks this token through a shared `requireAuth` middleware, which also re-fetches the farmer's current row so a deleted account is rejected immediately even if its old token has not expired.

6.2 Reference Data Module (Crops & Mandis)

Serves the list of crops and mandis the rest of the app is built on. Mandis can be bulk-imported from real government market names with only district and state — the backend looks up each mandi's real latitude/longitude through OpenWeatherMap's geocoding API automatically, rather than requiring coordinates to be typed in by hand. The mandi list also returns a live `farmersWaiting` count per mandi, computed from real confirmed bookings.

6.3 Live Price & Weather Module

Fetches real, current mandi commodity prices from the data.gov.in open government dataset for a given crop/mandi pair (latest price and MSP), a price-history view that groups the same feed's records by date, and real current weather for a mandi's coordinates from OpenWeatherMap. No price or weather figure in

this module is hardcoded or simulated.

6.4 Booking & Queue Module

Creates a booking with a unique token, and computes a farmer's live queue position by ranking all still-"CONFIRMED" bookings at the same mandi by slot date and then creation time — recomputed fresh on every request rather than stored, so it always reflects the current real state. Status updates are validated against a fixed set of six real stages before being applied.

6.5 Grievance Module

Lets a farmer file a complaint (optionally linked to a specific booking) and get back a real ticket number, then look up that ticket's status later. Status transitions are validated the same way as booking status.

6.6 AI Chatbot Module

A conversational assistant, powered by the Google Gemini API with function calling, that can act on a farmer's real data rather than just answering in text. On each message, the backend gives Gemini the real current crop list, mandi list, and the farmer's own last five bookings as context, then lets the model call real backend functions (looping up to 5 times per message) before replying:

Function	What it actually does
createBooking	Creates a real booking — only after the farmer has explicitly confirmed crop, quantity, mandi, and date
checkBookingStatus	Looks up the real current status of one of the farmer's bookings
checkQueueStatus	Looks up the farmer's real live queue position
fileGrievance	Files a real grievance and returns a real ticket number
checkGrievanceStatus	Looks up a real grievance's current status by ticket number
getCropPrice	Fetches the real live price for a crop at a mandi (data.gov.in)
getMandiWeather	Fetches the real live weather at a mandi (OpenWeatherMap)

The assistant replies in whichever language the farmer writes in, and is explicitly instructed to restate booking details and ask for confirmation before creating a booking, rather than acting silently.

7. API Reference

All endpoints are served from the Express backend. Endpoints marked “Yes” under Auth require a valid Authorization: Bearer <token> header.

Method	Endpoint	Auth	Description
POST	/auth/register	No	Create a farmer account; returns a JWT and the farmer profile
POST	/auth/login	No	Log in with phone + password; returns a JWT and the farmer profile
GET	/auth/me	Yes	Return the logged-in farmer's own profile
PUT	/auth/me	Yes	Update name, state, district, or preferred language
DELETE	/auth/me	Yes	Permanently delete the logged-in farmer's account
GET	/crops	No	List all crops
POST	/crops	No	Add a crop (name, slug, MSP)
GET	/mandis	No	List mandis, optionally filtered by state/district; includes live farmersWaiting count
POST	/mandis	No	Add a single mandi
POST	/mandis/bulk-import	No	Bulk-add mandis; auto-geocodes real coordinates, skips duplicates
GET	/mandis/regions	No	List real states and their districts, derived from saved mandis
GET	/mandis/:id/weather	No	Live current weather at a mandi's coordinates
GET	/mandis/:mandiId/crops/:cropId/price	No	Live latest price + MSP for a crop at a mandi
GET	/mandis/:mandiId/crops/:cropId/price-history	No	Real recent price points for a crop at a mandi, by date
POST	/chatbot/message	No*	AI chatbot turn; *farmerPhone identifies the farmer in the request body
POST	/bookings	No*	Create a booking; returns a unique token number
GET	/bookings	No*	List bookings, optionally filtered by farmerPhone
GET	/bookings/:id	No*	Get one booking with crop/mandi details
GET	/bookings/:id/queue	No*	Live queue position for a booking
POST	/bookings/:id/status	No*	Update a booking's status (validated against 6 real stages)
POST	/grievances	No*	File a grievance; returns a unique ticket number
GET	/grievances/:ticketNumber	No*	Look up a grievance by ticket number
POST	/grievances/:ticketNumber/status	No*	Update a grievance's status

* Booking, grievance, and chatbot endpoints identify the farmer by the phone number in the request body rather than a Bearer token, since these actions are also reachable through the public chatbot flow; the Profile and account-management endpoints are the ones gated behind login.

8. How MandiMitra Meets Each SIH Requirement

SIH requirement	MandiMitra feature	Status
Farmer registration and slot booking	Real farmer accounts (phone + password, state/district); booking flow for crop, mandi, quantity, and date	Implemented
Real-time queue management	Live queue-position endpoint and Track Queue page showing a farmer's position among that day's confirmed bookings at a mandi	Implemented
SMS/app notifications	In-app status visible on dashboard and through the AI chatbot; real SMS delivery not yet integrated	Partial — planned
Procurement and payment status tracking	Booking status field with six real stages: Confirmed → Arrived → Graded → Payment Initiated → Paid → Cancelled	Implemented
Reduced congestion and waiting time	Slot booking plus live queue visibility lets a farmer time their arrival instead of waiting physically at the mandi	Implemented by design

9. Feasibility and Viability

MandiMitra is not a mockup: it is a working system, deployed and reachable on the public internet, built entirely on free/low-cost managed infrastructure (Vercel, Railway, Supabase), which keeps running cost low enough for a pilot at a handful of mandis without dedicated infrastructure staff.

Real engineering challenges resolved during development:

- 1 Database reachability over IPv6.** Supabase's direct database connection requires IPv6, which the hosting platform did not support. Resolved by switching to Supabase's Transaction Pooler (IPv4-compatible).
- 2 Prisma vs. connection pooling.** Under real concurrent load, Prisma's default prepared-statement caching conflicted with the pooler sharing connections across requests ("prepared statement already exists"). Resolved by appending `?pgbouncer=true` to the database connection string, Prisma's documented setting for PgBouncer-style poolers.
- 3 Session stability.** Login sessions are signed with a JWT secret; keeping that secret fixed across deployments (rather than regenerating it) was necessary so farmers are not silently logged out whenever the backend redeploys.

Scalability: the backend is stateless (all session state lives in the JWT, not server memory), so it can be horizontally scaled behind a load balancer if adopted at a larger scale; the database is the natural next bottleneck and can be scaled independently on Supabase.

10. Testing and Validation

Testing was carried out directly against the live, deployed system rather than only locally, so the results below reflect real production behaviour.

Test	Method	Result
Database connectivity under load	8 sequential + 6 parallel real HTTP requests to a database-backed endpoint	14/14 succeeded (200 OK) after the PgBouncer fix; confirmed the earlier intermittent failure was resolved

Test	Method	Result
End-to-end registration → login	Registered a real farmer account through the live UI, confirmed dashboard access	Passed
Profile update	Edited name/state/district/language through the Profile page	Passed
Logout	Triggered from the Profile page and the navbar	Session cleared, redirected to login
Account deletion	Deleted a real test account, then attempted to log in with the same credentials	Login correctly rejected with “No account found” — confirmed real deletion, not a soft delete
SPA routing on direct load	Loaded a non-root route (e.g. /login) directly in a fresh browser tab	Passed after adding a Vercel rewrite rule for client-side routing

11. Impact and Benefits

For farmers:

- No more open-ended waiting at the mandi gate — a booked slot and a live queue position let them plan their arrival
- Visibility into MSP and live market prices before deciding where and when to sell
- Access in their own language (12 Indian languages supported), lowering the barrier for low-literacy or non-Hindi/English speaking farmers
- A single place to raise and track a grievance instead of an informal, undocumented complaint

For mandi administration / government:

- A real digital record of who is arriving, when, with how much produce — useful for staffing and capacity planning at procurement centres
- Fewer informal, in-person disputes over queue order, since position is tracked and visible to the farmer
- A foundation for data-driven policy: booking and grievance data is structured and queryable, unlike paper registers

Social and economic impact:

- Reduced time cost for farmers, who often lose a full working day waiting at procurement centres
- Greater trust in the procurement process through transparency of status and queue position
- A pattern that generalizes beyond one commodity or state, since crops, mandis, and states are all data-driven rather than hardcoded

12. Current Implementation Status

Stated honestly, as of this report:

Live and fully working today:

- Farmer registration and login (JWT-based, bcrypt-hashed passwords), including logout and permanent account deletion
- Slot booking by crop, mandi, quantity, and date
- Live queue-position tracking per mandi/day
- Booking status tracking through six real stages (Confirmed → Paid/Cancelled)
- Live crop prices and MSP lookup via the data.gov.in government API (on the Find Mandi page)
- Live weather lookup via OpenWeatherMap for a selected mandi
- AI chatbot (Gemini, function calling across 7 real functions) in the farmer's own language
- Grievance filing and tracking, linked to a booking
- Multilingual interface (12 Indian languages)
- Deployed and publicly reachable: frontend on Vercel, backend on Railway, database on Supabase

Known limitation: the dashboard's "farmer advisory" summary panel currently shows a small set of placeholder figures (explicitly labeled as a demo/sample in the interface) rather than a live feed; the same live weather and live price data is already real and working on the dedicated Find Mandi page and is the next candidate to also power that summary panel.

Not yet built (see Future Scope): real SMS delivery, an actual payment gateway, and a dedicated admin/mandi-staff interface for updating booking status (status changes currently happen through the

API).

13. Future Scope

- **Real SMS/WhatsApp notifications** via a telecom gateway (e.g. for booking confirmation, queue-turn alerts, payment confirmation) so farmers without a smartphone data connection still get updates
- **Payment gateway integration** to move the existing “payment initiated → paid” status from a manual/administrative update to an actual online payment flow
- **Admin / mandi-staff dashboard** for procurement centre staff to update booking status, manage daily capacity, and view the day's queue — today these transitions happen at the API level
- **Native mobile app** (the current app is a responsive web app, usable on a phone browser but not an installed app)
- **Offline/low-connectivity support**, since rural network coverage near mandis can be inconsistent
- **Analytics for policymakers**: aggregated, anonymized dashboards on wait times, procurement volumes, and grievance patterns across mandis

14. References

Resource	Link
data.gov.in — Variety-wise Daily Market Prices	https://www.data.gov.in/resource/9ef84268-d588-465a-a308-a864a43d0070
OpenWeatherMap API documentation	https://openweathermap.org/api
Google Gemini API documentation	https://ai.google.dev/gemini-api/docs
Prisma ORM documentation	https://www.prisma.io/docs
Supabase documentation	https://supabase.com/docs
Vercel documentation	https://vercel.com/docs
Railway documentation	https://docs.railway.com
React documentation	https://react.dev
Express.js documentation	https://expressjs.com

15. Team Details

Field	Value
Team name	[add here]
Institute	[add here]
Team leader	[add here]
Team members	[add here]